

Hard macro / fixed-block placement

Macros are hard fixed rectangles

Pseudocode

- Macro placement locks hard blocks first
- Free modules pack around those obstacles; legality fails if a macro drifts
- Open this module's examples file and find the Pseudocode section
- That written sketch is what you implement on the implement track and what the browser

Algorithm sketch

- Free golden has movable D at zero comma two
- Macro teaching pack pins D at zero comma zero with macro true and the rest legal

Algorithm sketch — try these

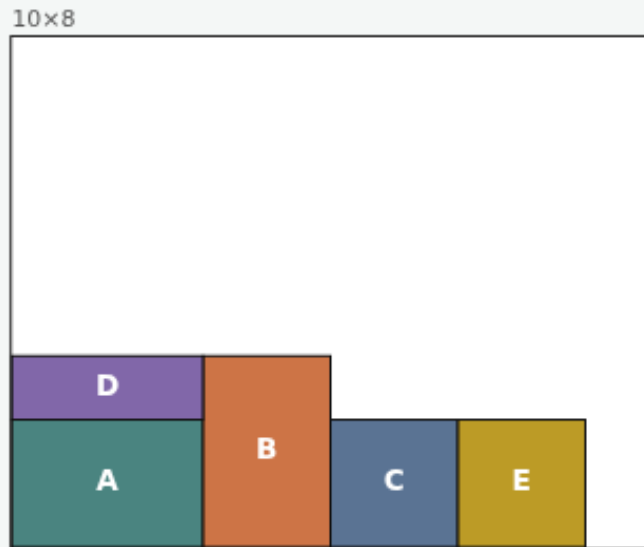
```
INPUT: macros F locked (x,y), free modules  
OUTPUT: legal pack; macros never move  
place each  $f \in F$  at locked pose (macro flag)  
pack free modules around F obstacles  
fail if any macro drifts  
GOLDEN free:  $D@(0,2)$ ; macro:  $D@(0,0)$ 
```

Free golden has movable D

MACRO PLACEMENT

Free golden has movable D

Step 1 / 5 · free · module03-05-macro-placement



Explanation

In the free golden packing, D sits at zero comma two above A. Nothing is flagged as a hard macro yet.

- D at (0,2)
- All blocks movable
- legal: true

mode: free
D@(0,2)

Free golden has movable D

Fix macro D at (0,0)

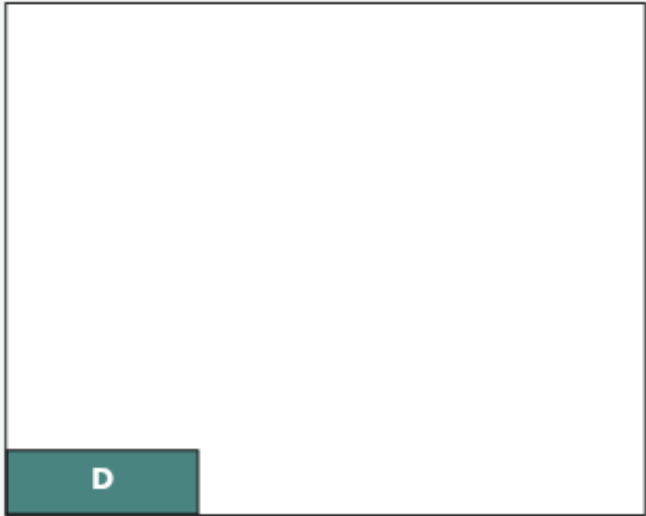
MACRO PLACEMENT

Fix macro D at (0,0)

Step 2 / 5 · fix-d · module03-05-macro-placement

10×8

D

A diagram showing a large rectangle representing a macro placement area, labeled '10x8' at the top-left corner. In the bottom-left corner of this rectangle, there is a smaller, dark green rectangle labeled 'D'. The 'D' is positioned at the origin (0,0) of the placement area.

Explanation

Macro mode pins hard block D at the origin and marks it macro true. Soft and standard cells must pack around that fixed rectangle.

- D fixed at (0,0)
- D.macro = true
- Size stays 3×1

```
D@(0,0)  
macro: true
```

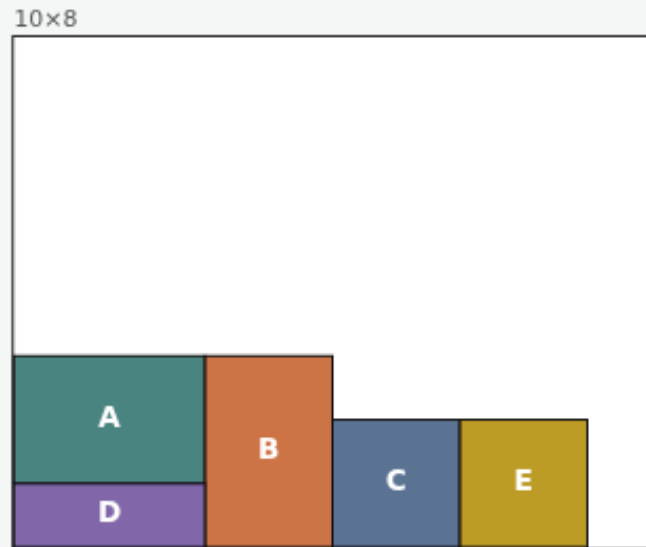
Fix macro D at (0,0)

Pack A-E around the macro

MACRO PLACEMENT

Pack A-E around the macro

Step 3 / 5 · pack-rest · module03-05-macro-placement



Explanation

A stacks above D; B, C, and E fill to the right. The macro packing differs from the free golden—D no longer sits at zero comma two.

- $A.y \geq D.h$
- Differs from free D position
- Five modules still present

```
A@(0,1)  
legal check next
```

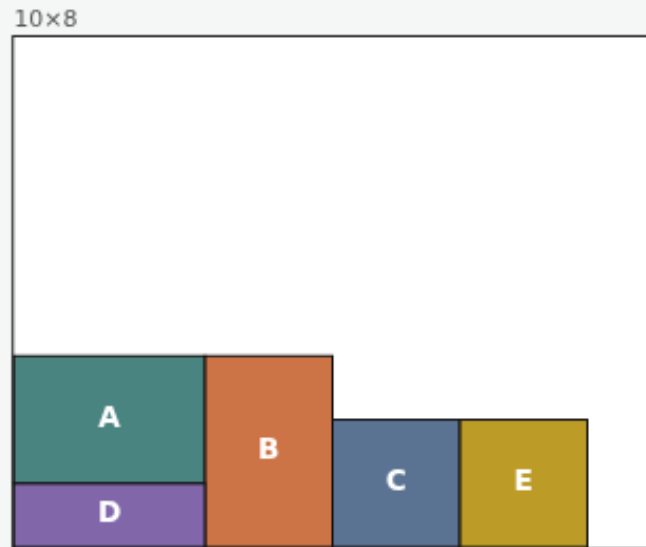
Pack A-E around the macro

Macro packing is legal

MACRO PLACEMENT

Macro packing is legal

Step 4 / 5 · legal · module03-05-macro-placement



Explanation

Despite the fixed block, the packing stays legal: no overflow, no overlaps. Fixed macros constrain search but do not excuse illegality.

- legal: true
- D remains 3x1
- Macro flag preserved

legal: true

Macro packing is legal

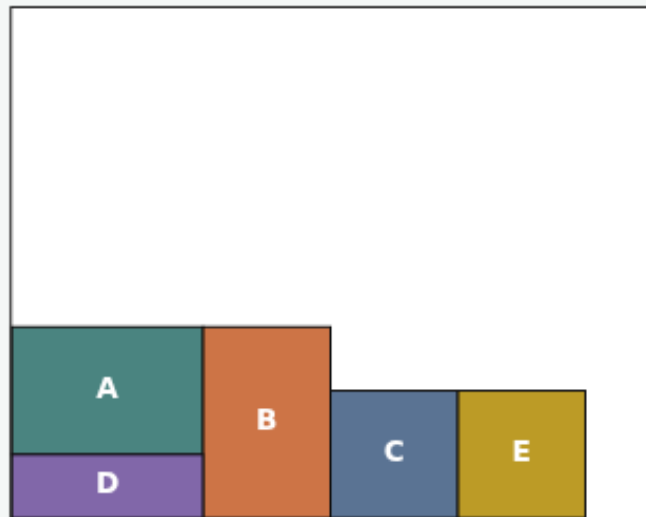
Macros first, then cells

MACRO PLACEMENT

Macros first, then cells

Step 5 / 5 · takeaway · module03-05-macro-placement

10×8



Explanation

Industrial flows often place large macros before standard cells. Treat fixed rectangles as hard constraints, then optimize the rest.

- Fix macros early
- Pack around them
- Next: hierarchical clusters

starter: D macro @ (0,0)

Macros first, then cells

Browser lab track

- Open macro-placement
- Compare free golden D at zero comma two with Place macros: D at zero comma zero
- Confirm legality and the macro flag

Implement track

- Fix D, pack the rest, assert D at (0,0), D.macro true, and is_legal_packing true
- Note the free golden differs

Pitfalls

- Moving macros after fixing them

Your turn

- Ship the legal macro packing
- Next: hierarchical AB left and CDE right at offset five